



Kernel

Ticks and Task Scheduling

This article explores how kernel functions and task scheduling are closely linked to time.

Time is vital for kernel programming since many kernel functions are time-driven. Some are periodic, such as push and pull migration for load-balancing, the scheduler that runs queues, or refreshing the screen. Their frequencies are fixed (such as 100 times per second). The kernel schedules other functions, such as delayed disk I/O, at a relative time in the future. For example, the kernel might schedule a floppy device driver to shut off after the floppy driver motor becomes inactive; this can be 50 milliseconds (say) from now or after completion of a certain task. So the kernel horology is relative. The kernel must also manage the system uptime, and the current date and time.

Events that occur periodically every 10 milliseconds are driven by the system timer. This is a programmable piece of hardware that issues an interrupt at a fixed frequency. The interrupt handler for this timer is called the *timer interrupt*. The hardware provides a system timer that the kernel uses to gauge the passing of time, which works off an electronic time source, such as a digital clock or the frequency of the processor. The system timer goes off (often called *hitting* or *popping*) at a pre-programmed frequency, called the *tick rate*. When the system timer goes off, it issues an interrupt that the kernel handles via a special interrupt handler. Because the kernel knows the pre-programmed tick rate, it knows the time between any two successive timer interrupts. This period is

called a *tick*. This is how the kernel keeps track of both ‘wall time’ and system uptime. ‘Wall time’, which is the actual time of day, is important to user-space applications. The kernel keeps track of it simply because the kernel controls the timer interrupt. The kernel defines the value in `<asm/param.h>`. For example, a microprocessor with an x86 architecture has 100 Hz, whereas one with the Intel Itanium architecture (earlier IA-64) has a 1024 Hz rate.

Timer interrupts

Interrupts are asynchronous events that are usually fired by external hardware; the CPU is interrupted in its current activity and executes special code—the ISR (Interrupt Service Routine)—to service the interrupt. Besides programming for hardware interrupts, the kernel services other interrupts. A module is expected to request an interrupt (or IRQ, for an interrupt request) channel before using it, and to release it when it’s done. The following functions, declared in `<linux/sched.h>` implement the interface:

```
int request_irq (
    unsigned int irq, //Interrupted number being requested
    void (*handler)(int, void *, struct pt_regs *), //function
    pointer to handle
    unsigned long flags,
```